czii-cryo-et-object-identification

3位解法

Rank	3
Team	ONCE UPON A MOON
Author	soso
Topic ID	561417
Version	Japanese Translation

Source: https://www.kaggle.com/competitions/czii-cryo-et-object-identification/discussion/561417
Retrieved with Kaggle CLI on 2026-10-21.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

この興味深いコンペを開催してくれた kaggle と主催者に感謝します。

このコンペで気に入っているのは、主催者がベースライン（特にデータ処理部分）を公開してくれたことです。私のようにこの分野の知識がない人にとって、とても役立ちました。

このコンペに参加したもう一つの理由は、ライブラリ accelerate に基づく学習コードを書いてみたかったことです（以前は純粋な pytorch を使っていました）。

上位に入れたので、私の学習コードには致命的なバグはないと考えることにします。

概要

このコンペを始める前は、@hengck23 が良い notebook を公開するまで、これはセグメンテーションではなく OD（物体検出）のタスクだと思っていました。

私の解法は、cc3d による後処理を使った 3D unet に基づいています。7種類の粒子すべてについて Cross Entropy loss を使っています。

最良の解法は、backbone res101 の 4 fold（7KF 中）の平均アンサンブルです。

一つ言及しておきたいのは、私の最良の解法は Public LB と Private LB の両方で同じスコア（0.783）だったことです。

モデル

segmentation_models_pytorch_3d のコードを使っています。

最良の解法は Unet + resnet101 です。

さまざまな backbone で別のアーキテクチャも試しましたが、public LB では unet+resnet101 が最良でした。

コードにはあまり深入りしなかったので、これらのモデルではほとんどデフォルトのパラメータを使いました。

学習

EMA を使いました。SWA の方が EMA より良いという話を聞いた覚えはありますが、EMA の方が扱いやすいからです。

モデルは入力サイズ (64, 128, 128) で学習しましたが、推論は (64, 256, 256) で行いました。これで public LB が 0.001 改善しました。

評価スコアの計算方法を考慮して、学習では元の半径の半分を使いました。これは私の実験でも最良でした。

データ拡張

データがとても少ないことを考えると、このコンペではデータ拡張が大いに役立つのは明らかです。

使ったもの:

- x、y、z 軸での反転
- x 軸と y 軸の入れ替え
- 異なるアルゴリズム: "denoised", "wbp", "ctfdeconvolved", "isonetcorrected
- 単純な copy past
- mixup

TTA

2つの tta を使い、出力を元の出力と平均しました。

- x、y、z の反転
- x、y の rot90

アンサンブル

- 7KF のうち 4 fold の平均アンサンブル

失敗

- 主催者が提供した外部データでの事前学習を試した
- resnet34 や resnet10 など、異なる backbone の unet とのアンサンブル

補足Q&A

AnnieGo — 質問

金メダル獲得おめでとうございます！私たちの7モデルのアンサンブルと同等の結果を、わずか4モデルで達成したのは見事です。各モデルの public と private のスコアを共有していただけますか。

soso — 回答

public 0.773、private 0.770 のモデルが1つあります。

Dive Deeper — 質問

おめでとうございます @tangtang1999。異なるアルゴリズム、mixup、copypaste のような強いデータ拡張を使ったことによる改善幅を教えていただけますか。

soso — 回答

残念ながらログが見つかりませんでした（ログが多すぎると notebook がとても遅くなるので、一部のログを削除しました 😅）。

覚えているのは次のことです。

- aug algo の確率を 0.5 から 0.75 に上げても、CV は良くならなかった
- copy past と mixup はどちらも CV を改善しなかったが、public LB を改善した。私は怠け者なので、1 fold（初期の実験では 5KF）しか提出していません
- その他については数値を覚えていませんが、CV が良くなったものだけを残しました。

このコンペでは学習データが少なすぎるので、ローカルの CV を常に信頼できるわけではないことを覚えておいてください

Leo Yang — 質問

こんにちは。解法を公開していただき、本当にありがとうございます！

私の理解では、あなたは「モデルを入力サイズ (64, 128, 128) で学習し、推論は (64, 256, 256) で行った」とのことです。学習と推論でサイズを変えると、どのような効果があるのでしょうか。この状況をどう理解すればよいでしょうか。

よろしくお願いします

Leo

soso — 回答

public LB で効いたからです。

小さなパッチで分割すると、モデルにとって予測が難しい境界のピクセルが増えるからだと思います！

Leo Yang — 質問

返信ありがとうございます。このケースにとても興味があります。もう少し質問してもよいでしょうか。

推論の入力サイズを変えた場合、結果は悪い影響を受けますか。

たとえば、2048*2048 ピクセルの解像度の画像を入力した場合、それでも正しくセグメンテーションできるでしょうか。

soso — 回答

- 推論の入力サイズを大きくすると、学習時との違いが大きくなるので、性能は下がるはずです。ただし、畳み込みモデルの受容野は固定なので、入力サイズが受容野より大きくなれば、それ以上は下がらないはずです。
- 入力サイズを大きくすると、より多くの文脈情報が得られるので、性能は上がるはずです。受容野については上と同じです。
- 入力サイズを大きくすると、境界のピクセルが減るので、性能は上がるはずです。

なので、何らかのバランスがあるはずです。